

Analysis of Algorithms (ENCT388) (Elective-II)

For BEI/BCT III-II

Tutorial 1 (Algorithm Analysis)

1. Write short notes on Best Case, Worst Case and Average case performance of an algorithm.
2. Explain RAM model in algorithm analysis.
3. What is recurrence relation? How it can be solved? Show that the time complexity of the recurrence relation $T(n) = 2T(n/2) + 1$ is $O(n)$ using substitution method.
4. Write short notes on Aggregate analysis.
5. Write short notes on time complexity and space complexity of an algorithm.
6. Write short notes on Big Oh notation with its mathematical definition and geometric interpretations.
7. Write short notes on Big Omega notation with its mathematical definition and geometric interpretations.
8. Write short notes on Big Theta notation with its mathematical definition and geometric interpretations.
9. The running time of an algorithm A is described by the recurrence relation

$$T(n) = 7T(n/2) + n^2$$

The competing algorithm B has a running time described by the recurrence relation

$$T(n) = aT(n/4) + n^2$$

What is the largest integer value for 'a' such that algorithm B is asymptotically faster than A?

10. Solve the following recurrence relations using Master's Method.
 - a) $T(n) = 9T(n/3) + n$
 - b) $T(n) = T(2n/3) + 1$
 - c) $T(n) = 3T(n/4) + n \lg n$
 - d) $T(n) = 2T(n/2) + n \lg n$
 - e) $T(n) = 2T(n/2) + \theta(n)$
 - f) $T(n) = 8T(n/2) + \theta(n^2)$
 - g) $T(n) = 7T(n/2) + \theta(n^2)$
 - h) $T(n) = 2T(n/4) + 1$
 - i) $T(n) = 2T(n/4) + \sqrt{n}$
 - j) $T(n) = 2T(n/4) + n$
 - k) $T(n) = 2T(n/4) + n^2$
11. Solve the following recurrence relations using recursion tree method. Assume $T(1) = 1$ and these recurrences are valid for $n > 1$
 - a) $T(n) = T(n/2) + 1$
 - b) $T(n) = 2T(n/2) + n$
 - c) $T(n) = 7T(n/2) + n^2$
 - d) $T(n) = 3T(n/4) + n$
 - e) $T(n) = 3T(n/2) + n$
 - f) $T(n) = 2T(n/3) + n$
 - g) $T(n) = 4T(n/2) + n^2$

h) $T(n) = T(n/2) + n$

i) $T(n) = 2T(n/2) + \lg n$

12. Given the following block of codes, write a recurrence relation for it and also find asymptotic upper bound (Assume that all dotted code takes constant time.)

```
Fun(int n)
{
... ..
if(condition1)
    x=Fun(n/4)
else if(condition2)
    x=Fun(n/2)
else
    x=Fun(3n/2)
... ..
}
```

13. Determine the time complexity and space complexity for the following block of codes:

a)

```
Algorithm Swap(p, q)
{
    temp=p;
    p=q;
    q=temp;
}
```

b)

```
Algorithm Sum(A, n)
{
    S=0;
    for(i=0;i<n;i++)
    {
        S=S+A[i];
    }
    return S;
}
```

c)

```
Algorithm Add(A, B, C, n)
{
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            C[i][j]=A[i][j]+B[i][j];
        }
    }
}
```

d)

```
Algorithm Multiply(A,B, C, n)
{
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            C[i][j]=0;
            for(k=0;k<n;k++)
            {
                C[i][j]+=A[i][k]*B[k][j];
            }
        }
    }
}
```

14. Find out the time complexity of following code snippets:

a)

```
void Test(int n)
{
    if(n>0)
    {
        printf("%d",n);
        Test(n-1);
    }
}
```

b)

```
void Test(int n)
{
    if(n>0)
    {
        for(i=0;i<n;i++)
        {
            printf("%d",n);
        }
        Test(n-1);
    }
}
```

c)

```
void Test(int n)
{
    if(n>0)
    {
        for(i=0;i<n;i=i*2)
        {
            printf("%d",i);
        }
        Test(n-1);
    }
}
```

d)

```
void Test(int n)
{
    if(n>1)
    {
        for(i=0;i<n;i++)
        {
            statement;
        }
        Test(n/2);
        Test(n/2);
    }
}
```

15. Show that the solution of $T(n) = T(n-1) + n$ is $O(n^2)$ using substitution method.
16. Show that the solution of $T(n) = 2T(n/2) + n$ is $O(n \lg n)$ using substitution method.
17. For each function $f(n)$ and time t in the following table, determine the largest size n of a problem that can be solved in time t , assuming that the algorithm to solve the problem take $f(n)$ microseconds.

	1 sec	1 minute	1 hour	1 day	1 month	1 year	1 century
$\lg n$							
$\sqrt{n}$							
n							
$n \lg n$							
n^2							
n^3							
2^n							
$n!$							